Leader Election

making sure exactly one node is in charge

Watch the 4-min video

In one line: leader election picks exactly one node to do a job or accept writes; do it with a lease in a consensus-backed store like etcd or ZooKeeper, renew it while you work, and use fencing tokens so a leader that lost its lease without knowing can't do damage.

1 Three instances, three charges

Your system runs a nightly billing job. For availability you deploy three instances. Now all three run the job, and every customer is charged three times. You need **exactly one** active instance, and if it dies, another must take over automatically. That's **leader election**.

2 Where you need a leader

- **Singleton jobs:** schedulers, cron-like tasks, cleanup workers.
- **Single-writer replication:** one primary database accepts writes (lesson 1).
- **Coordination:** Kafka's controller, Kubernetes controllers, a partition's leader.

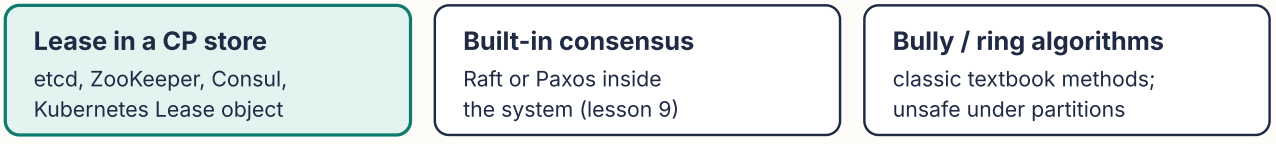


Figure 1. Ways to elect a leader. In practice, use a lease in a store that already runs consensus.

3 Leases: leadership with an expiry

A **lease** is leadership for a limited time. Candidates try to create a key in a strongly consistent store (etcd, ZooKeeper, Consul); the store guarantees only one succeeds. The winner must **renew** the lease before it expires (say, every 3 s for a 10 s lease). If it crashes or is cut off, the lease expires and the key is deleted; the others, which **watch** the key, race to become the next leader. Because the store itself runs Raft or Paxos, it never hands out two leases at once.

WORKED EXAMPLE: A BILLING SCHEDULER WITH ETCD

Three scheduler instances campaign through an etcd election with a 10-second lease. Instance 2 wins and runs the billing job, renewing its lease every few seconds. Instances 1 and 3 block on the election, watching the key. At 2:13 a.m. instance 2's machine dies. Its lease expires 10 s later, etcd deletes the key, instance 3 wins the next campaign and resumes the job from the last checkpoint. In Kubernetes, the same pattern uses a `Lease` object (client-go's leaderelection).

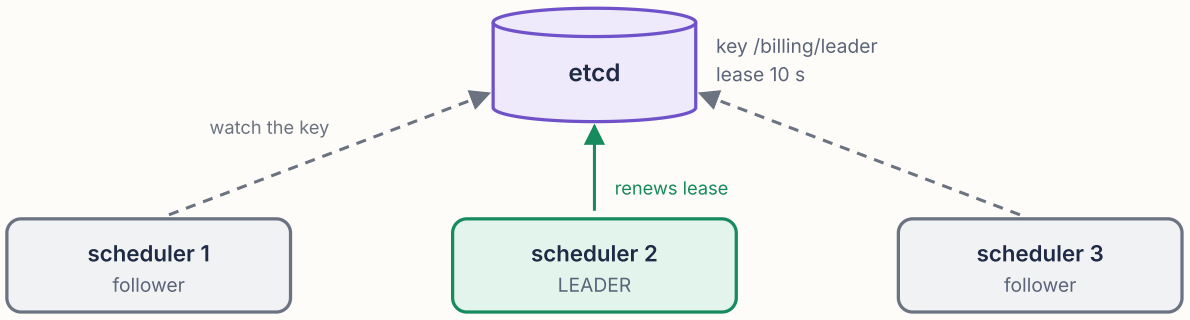


Figure 2. The leader holds a lease and renews it; followers watch the key and campaign when it disappears.

4 The zombie leader problem

A leader can be paused (a long GC, a frozen VM) past its lease. Everyone else has elected a new leader, but the old one wakes up still **believing** it's in charge and finishes its write. Checking the lease before writing doesn't help, because the pause can happen right after the check. The fix is a **fencing token**: a number that increases with every new lease (etcd's revision, ZooKeeper's zxid). The leader sends it with each write, and the storage rejects tokens older than the newest it has seen.

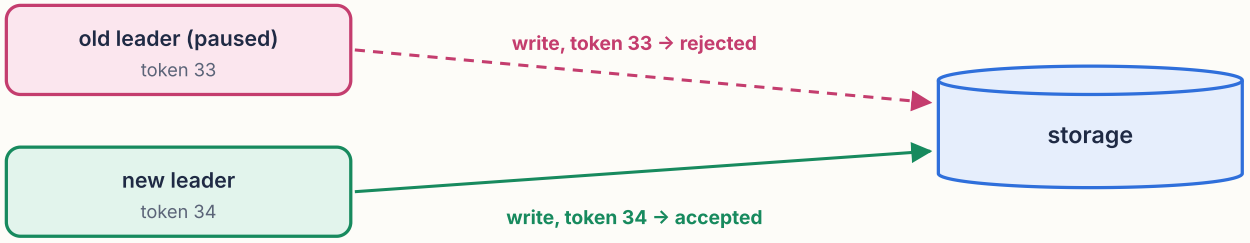


Figure 3. Fencing: the storage, not the leader, decides. Writes with an outdated token are rejected.

COMMON MISTAKES

- Electing a leader with a lock in a single Redis node, which can lose the lock on failover.
- No fencing, so a paused ex-leader still writes.
- Lease shorter than normal GC pauses, causing leadership flapping.
- Two-node clusters, where neither side can win a majority after a split.

5 Key terms

Leader: the one node allowed to do a job or accept writes

Campaign: trying to become leader

Fencing token: increasing number that lets storage reject an old leader

Lease: leadership that expires unless renewed

Watch: get notified when a key changes

Zombie leader: an old leader that doesn't know it lost

6 Interview questions

Q1. Why do you need leader election? BASIC

When exactly one node must perform a task or accept writes, like a scheduler or a primary database, and another must take over automatically if it fails.

Q2. How does a lease-based election work? BASIC

Candidates try to create a key with a lease in a consensus store; one wins and keeps renewing; if it stops, the lease expires, the key disappears, and the watchers campaign again.

Q3. Design failover for a singleton cron service on Kubernetes. SCENARIO

Run 2 to 3 replicas using the Kubernetes Lease API for election, make the job resumable with checkpoints, and pass a fencing token or use idempotent writes so a stale leader can't double-run work.

Q4. Your old leader paused for 20 s and then overwrote the new leader's data. Fix it. SCENARIO

Add fencing tokens from the lease revision and make the storage reject writes with older tokens; also lengthen the lease beyond normal pauses.

Q5. Why not elect a leader with a simple Redis SETNX? TRICKY

A single Redis node is a single point of failure, and with async replication a failover can lose the key, letting two nodes believe they lead. It also has no built-in fencing.

Q6. Can two leaders ever exist at the same time with leases? TRICKY

Briefly, yes: an old leader can believe it holds the lease while a new one is elected (pauses, clock drift). That's why safety must come from fencing at the resource, not from the leader's belief.

Next: SD-H-009 · Consensus and Raft backendroadmap.com